

Quiz Module Documentation

This document explains the **Model**, **Question Data**, and **UI** structure of the Flutter Quiz feature.

Question Model

File: `question_model.dart`

Purpose

Defines the structure of a quiz question. This model is reused across the app and keeps data consistent.

```
class Question {  
  final String question;  
  final List<String> options;  
  final int correctIndex;  
  
  Question({  
    required this.question,  
    required this.options,  
    required this.correctIndex,  
  });  
}
```

Fields

- `question` → The question text
 - `options` → List of answer options
 - `correctIndex` → Index of the correct option
-

Question Data

File: `quiz_questions.dart`

Purpose

Stores all quiz questions separately from UI logic. This makes the quiz **scalable** and easy to maintain.

```
final List<Question> questions = [  
  Question(  

```

```

    question: "Which one is Banana?",
    options: ["Apple", "fruit", "Orange", "Mango"],
    correctIndex: 1,
  ),
];

```

Benefits

- Easy to add/remove questions
- Can later load from JSON / API
- Keeps UI clean

Quiz UI (Full UI Code)

File: `quiz_page.dart`

This file contains the complete UI and interaction logic for the quiz screen. It: - Displays questions and options - Handles option selection - Shows correct / wrong colors - Moves to next question - Navigates to next page when quiz finishes

Full UI Implementation

```

import 'package:flutter/material.dart';
import 'question_model.dart';
import 'quiz_questions.dart';
import 'result_page.dart';

class QuizPage extends StatefulWidget {
  const QuizPage({super.key});

  @override
  State<QuizPage> createState() => _QuizPageState();
}

class _QuizPageState extends State<QuizPage> {
  int currentQuestionIndex = 0;
  int? selectedIndex;

  Question get currentQuestion => questions[currentQuestionIndex];

  Color getOptionColor(int index) {
    if (selectedIndex == null) {
      return const Color(0xFFFFFFFF);
    }
  }

```

```

    }

    if (index == currentQuestion.correctIndex) {
      return const Color(0xFFFF0FBF5);
    }

    if (index == selectedIndex &&
        selectedIndex != currentQuestion.correctIndex) {
      return const Color(0xFFFAE4E4);
    }

    return const Color(0xFFFFFFFF);
  }

  Color getOptionColorborder(int index) {
    if (selectedIndex == null) {
      return const Color(0xFFDFE0E4);
    }

    if (index == currentQuestion.correctIndex) {
      return const Color(0xFF2FA75F);
    }

    if (index == selectedIndex &&
        selectedIndex != currentQuestion.correctIndex) {
      return const Color(0xFFFF60000);
    }

    return const Color(0xFFDFE0E4);
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      body: Padding(
        padding: const EdgeInsets.all(16),
        child: Column(
          crossAxisAlignment: CrossAxisAlignment.start,
          children: [
            Text(
              currentQuestion.question,
              style: const TextStyle(
                fontSize: 20,
                fontWeight: FontWeight.bold,
              ),
            ),
            const SizedBox(height: 20),

```

```

...List.generate(currentQuestion.options.length, (index) {
  return GestureDetector(
    onTap: selectedIndex == null
      ? () => setState(() => selectedIndex = index)
      : null,
    child: Container(
      width: double.infinity,
      margin: const EdgeInsets.only(bottom: 12),
      padding: const EdgeInsets.all(16),
      decoration: BoxDecoration(
        border: Border.all(
          color: getOptionColorborder(index),
          width: 1.5,
        ),
        color: getOptionColor(index),
        borderRadius: BorderRadius.circular(12),
      ),
      child: Text(
        currentQuestion.options[index],
        style: const TextStyle(fontSize: 16),
      ),
    ),
  );
}),

const Spacer(),

 SizedBox(
  width: double.infinity,
  height: 50,
  child: ElevatedButton(
    onPressed: selectedIndex == null
      ? null
      : () {
        if (currentQuestionIndex < questions.length - 1) {
          setState(() {
            currentQuestionIndex++;
            selectedIndex = null;
          });
        } else {
          Navigator.pushReplacement(
            context,
            MaterialPageRoute(
              builder: (_) => const ResultPage(),
            ),
          );
        }
      },
  ),
),

```

```
child: const Text(  
  "Next",  
  style: TextStyle(fontSize: 18),  
),  
),  
),  
],  
),  
),  
);  
}  
}
```

UI Behavior Summary

- One option selectable per question
- Correct answer always highlighted
- Wrong answer highlighted only if selected
- Next button disabled until selection
- Navigates to next page after last question

 This UI section shows the **complete and exact implementation** used in the quiz screen.